07 - Herencia y polimorfismo

Pregunta número 1

¿Qué imprime este programa de herencia?

```
class Padre {
    public Padre() {
        System.out.println("Clase Padre");
    }
}
class Hija extends Padre {
    public Hija() {
        System.out.println("Clase Hija");
    }
}
public class Test {
    public static void main(String[] args) {
        new Hija();
    }
}
```

Seleccione una:

- a. Error de compilación
- b. Clase Hija
- c. Clase Hija Clase Padre
- d. Clase Padre Clase Hija

Respuesta: Lo primero que debemos recordar es que cuando creamos un objeto de una subclase, Java siempre construye primero la parte que hereda de la superclase. Aunque el constructor de Hija no escribe super() de forma explícita, el compilador lo inserta automáticamente como primera instrucción. Así que, al ejecutar new Hija(), lo primero que corre es el constructor de Padre, que imprime "Clase Padre", y solo cuando termina, se ejecuta el resto del constructor de Hija, que imprime "Clase Hija". Me gusta pensarlo como construir una casa: primero van los cimientos (la clase padre) y después los pisos de arriba (la clase hija). El código compila sin problemas, porque Padre tiene un constructor sin argumentos que el super() implícito puede llamar, y el orden inverso del inciso c nunca podrá ocurrir.

Pregunta número 2

Given the following classes, which line of code correctly creates an instance of the Dog class?

```
class Animal {
    // ...
}
class Dog extends Animal {
    // ...
}
```

Seleccione una o más de una:

- [x] a. Dog dog = new Dog();
- [x] b. Animal dog = new Dog();
- [ ] c. Dog dog = new Animal();
- [ ] d. Animal dog = new Animal();

Respuesta: La clave de esta pregunta está en leer con cuidado lo que nos están pidiendo, ya que no basta con que la línea compile, sino que debe crear una instancia de la clase Dog. Y lo que define qué objeto se crea no es el tipo de la variable, sino lo que va después de new. El inciso a es la forma más directa: new Dog() crea un objeto Dog y lo guarda en una referencia del mismo tipo. El inciso b también crea un objeto Dog, solo que lo guarda en una referencia de tipo Animal; esto es completamente válido, porque un perro “es un” animal, y es justamente la base del polimorfismo. El inciso c no compila, ya que intenta guardar un objeto de la superclase en una referencia de la subclase, y no todo animal es un perro. El inciso d es la trampa más fina: sí compila, porque Animal no es abstracta y la referencia es del mismo tipo que el objeto, pero lo que crea es un Animal, no un Dog. Que la variable se llame dog no cambia nada, ya que el nombre de una variable es solo una etiqueta y no tiene ningún efecto sobre el tipo del objeto. Por eso, las únicas dos líneas que crean una instancia de Dog son la a y la b.

Pregunta número 3

Which statement, when inserted into line '' // TODO code application logic here", is valid in compilation time change?

```
public class SampleClass {
    public static void main(String[] args) {
        AnotherSampleClass asc = new AnotherSampleClass();
        SampleClass sc = new SampleClass();
        // TODO code application logic here
    }
}
class AnotherSampleClass extends SampleClass { }
```

Seleccione una:

- a. asc = sc;
- b. sc = asc;
- c. asc = (Object) sc;
- d. asc= sc.clone();

Respuesta: Siento yo que el punto clave de este ejercicio es que tenemos dos referencias: asc, de tipo AnotherSampleClass (la subclase), y sc, de tipo SampleClass (la superclase). La asignación sc = asc es válida porque se trata de una conversión hacia arriba (upcasting): un objeto de la subclase siempre "es un" objeto de la superclase, y el compilador la acepta sin necesidad de ningún cast. El inciso a intenta lo contrario, asignar una superclase a una subclase sin cast, y eso no compila. El inciso c empeora la situación, porque convierte sc a Object y luego intenta guardar un Object en una referencia AnotherSampleClass, lo cual tampoco compila. El inciso d tiene dos problemas: clone() es protected en Object, y aunque aquí sería accesible por estar en la misma jerarquía, regresa un Object, que no podemos asignar a asc sin un cast. Por eso, la única instrucción válida en tiempo de compilación es sc = asc.

Pregunta número 4

¿Cuál será el resultado de ejecutar el siguiente código?

```
class Parent {
    public void display() {
        System.out.println("Parent");
    }
}
class Child extends Parent {
    public void display() {
        System.out.println("Child");
    }
}
public class Test {
    public static void main(String[] args) {
        Parent obj = new Child();
        obj.display();
    }
}
```

Seleccione una:

- a. Parent
- b. Error de ejecución
- c. Child
- d. Error de compilación

Respuesta: En este ejercicio podemos ver el polimorfismo en acción. La referencia obj es de tipo Parent, pero el objeto que realmente se crea en memoria es un Child. Cuando llamamos a obj.display(), el compilador verifica que el método exista en el tipo de la referencia (Parent sí tiene display()), y por eso compila. Pero en tiempo de ejecución, la JVM no se fija en el tipo de la referencia, sino en el tipo real del objeto, y como Child sobrescribe display(), se ejecuta su versión, que imprime "Child". A esto se le conoce como enlace dinámico (dynamic binding). El inciso a es la trampa para quien piense que manda el tipo de la referencia; eso solo ocurre con los atributos y los métodos static, no con los métodos de instancia sobrescritos.

Pregunta número 5

¿Cuál será el resultado del siguiente fragmento de código?

```
class Animal {
    public void makeSound() {
        System.out.println("Animal sound");
    }
}

class Dog extends Animal {
    @Override
    public void makeSound() {
        System.out.println("Bark");
    }
}

public class Test {
    public static void main(String[] args) {
        Animal myDog = new Dog();
        myDog.makeSound();
    }
}
```

Seleccione una:

- a. Animal sound
- b. Error de ejecución
- c. Error de compilación
- d. Bark

Respuesta: Este ejercicio refuerza la idea de la pregunta anterior. Declaramos myDog como Animal, pero el objeto real es un Dog. El método makeSound() existe en Animal, así que la llamada compila, y la anotación @Override en Dog confirma que la estamos sobrescribiendo correctamente: misma firma, mismo tipo de retorno y el mismo nivel de acceso público. Al ejecutarse, la JVM busca la versión del método en la clase real del objeto, que es Dog, y por eso se imprime "Bark". Si quisiéramos que se ejecutara la versión de Animal, tendríamos que crear un objeto

de tipo Animal o, desde dentro de Dog, llamar a super.makeSound(). No hay ningún error de compilación ni de ejecución, por lo que los incisos b y c quedan descartados.

Pregunta número 6

¿Cuál será la salida del siguiente código?

```
class Animal {
    public void sonido() {
        System.out.println("Sonido genérico de animal.");
    }
}

class Perro extends Animal {
    @Override
    public void sonido() {
        System.out.println("Ladrido.");
    }
}

public class TestHerencia {
    public static void main(String[] args) {
        Animal miAnimal = new Perro();
        miAnimal.sonido();
    }
}
```

Seleccione una:

- a. Sonido genérico de animal.
- b. Ladrido.
- c. Error de compilación.
- d. No se imprime nada.

Respuesta: Para mí, la forma más sencilla de resolver estas preguntas es preguntarme siempre qué objeto se creó con new, porque ese es el que decide qué método se ejecuta. Aquí la variable miAnimal es de tipo Animal, pero el objeto es un

Perro. Perro sobrescribe el método sonido() con la anotación @Override, por lo que en tiempo de ejecución, la JVM ejecuta la versión de Perro y se imprime "Ladrido.". El tipo de la referencia solo sirve para que el compilador sepa qué métodos podemos llamar; como Animal también tiene sonido(), la llamada es válida y el código compila. El inciso a sería el resultado si no hubiera sobrescritura, y el inciso d no tiene sentido porque el método sí imprime algo.

Pregunta número 7

Que muestra en pantalla al ejecutar este código?

```
class Parent {
    public void display() {
        System.out.println("Parent");
    }
}
class Child extends Parent {
    public void display() {
        System.out.println("Child");
    }
}
public class Test {
    public static void main(String[] args) {
        Parent obj = new Child();
        ((Child) obj).display();
    }
}
```

Seleccione una:

- a. Child
- b. Parent
- c. Compilation error
- d. Runtime error

Respuesta: En este ejercicio tenemos un cast explícito hacia abajo (downcasting): ((Child) obj).display(). La referencia obj es de tipo Parent, pero el objeto real que creamos es un Child, así que el cast es seguro; la JVM verifica en tiempo de

ejecución que el objeto realmente sea un Child, y como sí lo es, no lanza ninguna ClassCastException. Lo interesante es que el cast no cambia el resultado: aunque no lo hubiéramos puesto, obj.display() también ejecutaría la versión de Child por el polimorfismo. Con o sin cast, el método que se ejecuta depende del objeto, y por eso se imprime "Child". El cast solo sería necesario si quisiéramos llamar a un método que existiera únicamente en Child. El inciso d sería correcto únicamente si el objeto creado fuera un Parent puro, porque entonces el cast fallaría.

Pregunta número 8

¿Cuál es el resultado del siguiente código? (SampleClass y AnotherSampleClass están en el paquete por defecto y AnotherSampleClass extends SampleClass.)

```
SampleClass sc = new SampleClass();
AnotherSampleClass asc = new AnotherSampleClass();
System.out.println("sc: " + sc.getClass());
System.out.println("asc: " + asc.getClass());
```

Seleccione una:

- a. sc: class Object asc: class AnotherSampleClass
- b. sc: class SampleClass asc: class AnotherSampleClass
- c. sc: class AnotherSampleClass asc: class SampleClass
- d. sc: class AnotherSampleClass asc: class AnotherSampleClass

Respuesta: Lo primero que noté es que el método getClass() regresa la clase real del objeto en tiempo de ejecución, y cuando lo concatenamos con un String, se imprime con el formato "class NombreDeLaClase". En este caso, sc es una referencia SampleClass que apunta a un objeto creado con new SampleClass(), y asc es una referencia AnotherSampleClass que apunta a un objeto creado con new AnotherSampleClass(). Como cada referencia apunta a un objeto de su propio tipo, se imprime "sc: class SampleClass" y "asc: class AnotherSampleClass". Como ambas clases están en el paquete por defecto, no aparece ningún nombre de paquete antes del nombre de la clase. El inciso a es incorrecto porque getClass() nunca regresa Object para un objeto de otra clase, y los incisos c y d invierten o mezclan los tipos.

Pregunta número 9

What is the result?

```
1  public class SampleClass {
2      public static void main(String[] args) {
3          AnotherSampleClass asc = new AnotherSampleClass();
4          SampleClass sc = new SampleClass();
5          sc = asc;
6          System.out.println("sc: " + sc.getClass());
7          System.out.println("asc: " + asc.getClass());
8      }
9  }
10
11 class AnotherSampleClass extends SampleClass {}
```

Seleccione una:

- a. sc: class SampleClass asc: class AnotherSampleClass
- b. sc: class Object asc: class AnotherSampleClass
- c. sc: class AnotherSampleClass asc: class SampleClass
- d. sc: class AnotherSampleClass asc: class AnotherSampleClass

Respuesta: Esta pregunta es casi igual a la anterior, pero con una línea extra que cambia todo: en la línea 5 se ejecuta sc = asc. A partir de ese momento, ambas referencias apuntan al mismo objeto, que fue creado con new AnotherSampleClass(). El objeto SampleClass original que se creó en la línea 4 queda sin ninguna referencia y será candidato para el recolector de basura. Recordemos que getClass() no se fija en el tipo de la variable, sino en la clase real del objeto, por lo que tanto sc.getClass() como asc.getClass() regresan class AnotherSampleClass. El inciso a es la trampa para quien piense que getClass() depende del tipo declarado de la variable. La asignación de la línea 5 compila sin problemas, porque es un upcasting de la subclase a la superclase.

Pregunta número 10

¿Cuál es el resultado del siguiente código?

```
public class SampleClass {
    public static void main(String[] args) {
        SampleClass sc, scA, scB;
        sc = new SampleClass();
        scA = new SampleClassA();
        scB = new SampleClassB();
        System.out.println("Hash is: " + sc.getHash() + ", " +
                scA.getHash() + ", " + scB.getHash());
    }

    public int getHash() {
        return 111111;
    }
}

class SampleClassA extends SampleClass {
    @Override
    public int getHash() {
        return 44444444;
    }
}

class SampleClassB extends SampleClass {
    @Override
    public int getHash() {
        return 999999999;
    }
}
```

Seleccione una:

- a. La compilación falla.
- b. Se lanza una excepción en tiempo de ejecución.
- c. Hash is: 111111, 44444444, 999999999
- d. No hay resultado porque este no es el método correcto para determinar el hash.

Respuesta: Lo primero y, a mi parecer, el punto clave es que las tres variables se declaran del tipo SampleClass, pero cada una apunta a un objeto distinto: sc a un SampleClass, scA a un SampleClassA y scB a un SampleClassB. Como ambas

subclases sobrescriben getHash() (y el @Override lo confirma), al llamar al método se ejecuta la versión del objeto real gracias al polimorfismo. Por eso, sc.getHash() regresa 111111, scA.getHash() regresa 44444444 y scB.getHash() regresa 999999999. Este último valor cabe sin problema en un int, ya que el máximo es 2,147,483,647, así que no hay desbordamiento ni error de compilación. El inciso d es un distractor que juega con el nombre del método: getHash() no es hashCode(), es simplemente un método común que nosotros definimos, así que regresa lo que nosotros le indiquemos.

Pregunta número 11

What will be the output of the following code snippet?

```
class A {
    public A() {
        System.out.println("A's constructor");
    }
}
class B extends A {
    public B() {
        super();
        System.out.println("B's constructor");
    }
}
class C extends B {
    public C() {
        System.out.println("C's constructor");
    }
}
public class Test {
    public static void main(String[] args) {
        C c = new C();
    }
}
```

Seleccione una:

- a. A's constructor B's constructor C's constructor
- b. C's constructor A's constructor B's constructor
- c. A's constructor C's constructor B's constructor

Respuesta: En este ejercicio, lo primero que noté es que tenemos una cadena de herencia de tres niveles: C extiende de B y B extiende de A. Cuando ejecutamos new C(), el constructor de C llama implícitamente a super(), que lleva al constructor de B. B tiene un super() explícito como primera línea, que lleva al constructor de A. A es el primero en terminar de ejecutarse e imprime "A's constructor". Luego, se regresa a B, que imprime "B's constructor", y finalmente se regresa a C, que imprime "C's constructor". La clave es que las llamadas a los constructores suben por la jerarquía, pero los cuerpos se ejecutan de arriba hacia abajo, desde la superclase más alta hasta la subclase. Que B escriba super() de forma explícita y C no, no cambia nada, ya que el compilador lo agrega automáticamente si no lo ponemos.

Pregunta número 12

Que imprime en pantalla este código?

```
class Base {
    public Base() {
        System.out.println("Base constructor");
    }
    public Base(String message) {
        System.out.println("Base constructor with message: " + message);
    }
}
class Derived extends Base {
    public Derived() {
        super("Hello");
        System.out.println("Derived constructor");
    }
}
public class Test {
    public static void main(String[] args) {
        Derived derived = new Derived();
    }
}
```

Seleccione una:

- a. It will print "Base constructor" followed by "Derived constructor"
- b. It will print "Base constructor with message: Hello" followed by "Derived constructor"
- c. It will throw a compilation error.
- d. It will print "Derived constructor" followed by "Base constructor with message: Hello"

Respuesta: El punto clave de esta pregunta, a mi parecer, es notar que la clase Base tiene dos constructores: uno sin argumentos y otro que recibe un String. El constructor de Derived llama de forma explícita a super("Hello"), y ese detalle decide

cuál de los dos se ejecuta. Como le pasamos un String, se ejecuta el constructor con parámetro, que imprime "Base constructor with message: Hello". El constructor sin argumentos nunca se llama, porque el compilador solo inserta un super() implícito cuando no escribimos ninguna llamada a super() ni a this(). Al terminar el constructor de Base, se ejecuta el resto del constructor de Derived, que imprime "Derived constructor". El inciso a es la trampa para quien piense que siempre se ejecuta el constructor por defecto del padre, y el inciso d invierte el orden.

Pregunta número 13

What is the result?

```
class Alpha { String getType() { return "alpha"; } }
class Beta extends Alpha { String getType() { return "beta"; } }
public class Gamma extends Beta { String getType() { return "gamma"; }
    public static void main(String[] args) {
        Gamma g1 = new Alpha();
        Gamma g2 = new Beta();
        System.out.println(g1.getType() + " " + g2.getType());
    }
}
```

Seleccione una:

- a. Alpha beta
- b. Beta beta.
- c. Gamma gamma.
- d. Compilation fails.

Respuesta: En este ejercicio puede ocurrir que solo nos fijemos en los métodos getType(); esto nos podría llevar a pensar en el polimorfismo, pero el problema está antes, en las asignaciones. La jerarquía es: Gamma extiende de Beta y Beta extiende de Alpha, así que Gamma es la subclase más baja. En el main se intenta hacer Gamma g1 = new Alpha() y Gamma g2 = new Beta(), es decir, guardar objetos de las superclases en una referencia de la subclase. Esto no se puede hacer, porque un Alpha no necesariamente es un Gamma, y el compilador marca un error de tipos incompatibles en ambas líneas. Ni siquiera con un cast funcionaría en tiempo de ejecución, porque los objetos realmente no son de tipo Gamma. Por eso, la compilación falla y ninguna de las salidas de los otros incisos es posible.

Pregunta número 14

What will happen when you try to compile the following code?

```
class Parent {
    protected void show() {
        System.out.println("Parent show");
    }
}
class Child extends Parent {
    @Override
    void show() {
        System.out.println("Child show");
    }
}
public class Test {
    public static void main(String[] args) {
        Child child = new Child();
        child.show();
    }
}
```

Seleccione una:

- a. It will print "Parent show".
- b. It will print "Child show".
- c. Compilation error.
- d. It will print "Parent show" followed by "Child show".

Respuesta: En este ejercicio debemos recordar una de las reglas más importantes de la sobrescritura y esa regla es que el método de la subclase no puede tener un modificador de acceso más restrictivo que el método original. En Parent, show() es protected, y en Child se intenta sobrescribir con el acceso por defecto (package-private), que es más restrictivo que protected. Recordemos el orden de menor a mayor restricción: public, protected, default y private. Como estamos reduciendo la visibilidad, el compilador marca un error que dice que show() en Child no puede sobrescribir show() en Parent porque intenta asignar privilegios de acceso más débiles. Para corregirlo, tendríamos que declarar el método en Child como

protected o como public. Al no compilar, ninguna de las salidas de los demás incisos ocurre.

Pregunta número 15

What is the result?

```
class CA {
    CA() {
        System.out.print("middle");
    }
    CA(String s) {
        System.out.print(s);
    }
}
class CB extends CA {
}
public class Test {
    public static void main(String[] args) {
        new CA("message ");
        new CB();
        System.out.print(" end");
    }
}
```

Seleccione una:

- a. message middle end
- b. An exception is thrown at runtime.
- c. Compilation fails
- d. message end middle

Respuesta: La clave de este ejercicio es seguir el main línea por línea. Primero, new CA("message ") llama al constructor de CA que recibe un String e imprime "message ". Después viene new CB(). La clase CB no declara ningún constructor, así que el compilador le crea uno por defecto, sin argumentos, que lo único que hace es llamar a super(). Ese super() ejecuta el constructor sin argumentos de CA, que imprime "middle". Finalmente, se imprime " end". La salida completa es

"message middle end". El código compila perfectamente porque CA tiene un constructor sin argumentos que el constructor por defecto de CB puede llamar. Si CA solo tuviera el constructor con String, entonces sí tendríamos un error de compilación en CB, que es justo lo que intenta sugerir el inciso c.

Pregunta número 16

Which three methods, inserted individually at line, will correctly complete class Two(Choose three)?

```
10.    class One {
11.        void foo() { }
12.    }
13.    class Two extends One {
14.        //insert method here
15.    }
```

Seleccione una o más de una:

- [ ] a. int foo() { /* more code here */ }
- [x] b. void foo() { /* more code here */ }
- [x] c. public void foo() { /* more code here */ }
- [ ] d. private void foo() { /* more code here */ }
- [x] e. protected void foo() { /* more code here */ }

Respuesta: Primero que nada, en la clase One, foo() tiene acceso por defecto (package-private) y regresa void. Para sobrescribirlo en Two, debemos respetar la misma firma y el mismo tipo de retorno, y no podemos reducir la visibilidad, aunque sí podemos aumentarla. El inciso b es válido porque mantiene exactamente el mismo acceso por defecto. El inciso c es válido porque public es más visible que el acceso por defecto, y el inciso e también lo es, ya que protected es más amplio que el acceso por defecto (permite el acceso desde el paquete y desde subclases). El inciso a falla porque cambia el tipo de retorno a int con la misma lista de parámetros; eso no es una sobrecarga válida, ya que la sobrecarga exige parámetros distintos, y tampoco es una sobrescritura válida, porque int no es compatible con void. El inciso d falla porque private es más restrictivo que el acceso por defecto.

Pregunta número 17

¿Qué métodos de anulación (Override) son válidos para las siguientes clases?

```
class Parent {
    void display() {
        System.out.println("Parent");
    }
}
class Child extends Parent {
    // Override here
}
```

Seleccione una o más de una:

- [ ] a. static void display() { System.out.println("Child"); }
- [ ] b. private void display() { System.out.println("Child"); }
- [x] c. void display() { System.out.println("InChild"); }
- [x] d. public void display() { System.out.println("Child"); }

Respuesta: El método display() de Parent tiene acceso por defecto y no es static, así que para sobrescribirlo en Child debemos mantener la misma firma y el mismo tipo de retorno, y usar el mismo nivel de acceso o uno más amplio. El inciso c es válido porque mantiene el acceso por defecto; que imprima "InChild" en lugar de "Child" es irrelevante, ya que el contenido del método puede ser lo que queramos. El inciso d también es válido, ya que public es más amplio que el acceso por defecto. El inciso a falla porque un método static no puede ocultar (hide) a un método de instancia de la superclase; el compilador lo rechaza. El inciso b falla porque private es más restrictivo que el acceso por defecto, y eso reduce la visibilidad del método heredado.

Pregunta número 18

Which line of code, when inserted at the commented line, causes the compiler to fail?

```
class ABC {
    public String met() {
        return "snippet 1";
    }
}
public class ABCD extends ABC {
    // Insert code here
}
```

Seleccione una:

- a. public String met() { return "4"; }
- b. public char met(int i) { return '3';}
- c. public String met(int i) { return "5"; }
- d. public char met() { return '2'; }

Respuesta: En esta pregunta tenemos que distinguir entre sobrescritura y sobrecarga. La sobrescritura ocurre cuando el método de la subclase tiene exactamente la misma lista de parámetros que el de la superclase, y en ese caso el tipo de retorno debe ser el mismo o un subtipo (retorno covariante). La sobrecarga ocurre cuando la lista de parámetros es distinta, y ahí el tipo de retorno puede ser cualquiera. El inciso d, public char met(), tiene la misma firma que met() de ABC (sin parámetros), por lo que es una sobrescritura, pero cambia el tipo de retorno de String a char, que no es compatible, así que el compilador falla. El inciso a es una sobrescritura válida, ya que tiene la misma firma y el mismo tipo de retorno. Los incisos b y c reciben un parámetro int, así que son sobrecargas, y en ese caso el tipo de retorno puede ser char o String sin problema.

Pregunta número 19

¿Cuál de las siguientes afirmaciones describe con precisión las relaciones que pueden existir entre clases en Java?

Seleccione una o más de una:

- [x] a. La herencia representa una relación "es-un" donde una clase deriva de otra clase.
- [x] b. La composición representa una relación "tiene-un" donde una clase contiene una instancia de otra clase.
- [ ] c. Se debe preferir la herencia sobre la composición para fomentar la reutilización de código y la flexibilidad.
- [x] d. Tanto la herencia como la composición pueden usarse juntas para modelar relaciones complejas.
- [x] e. El uso (o asociación) representa una relación "usa-un" donde una clase utiliza métodos o instancias de otra clase.
- [x] f. Se debe preferir la composición sobre la herencia para fomentar la reutilización de código y la flexibilidad.

Respuesta: La clave de esta pregunta es revisar cada afirmación sobre las relaciones entre clases. El inciso a es correcto, ya que la herencia representa una relación "es-un"; por ejemplo, un Perro es un Animal. El inciso b también lo es, ya que la composición representa una relación "tiene-un", donde una clase contiene una instancia de otra; por ejemplo, un Auto tiene un Motor. El inciso d es verdadero, porque en el mundo real es común combinar ambas; por ejemplo, un AutoDeportivo puede heredar de Auto y, al mismo tiempo, tener un Motor. El inciso e también es correcto, ya que la asociación o relación de uso ("usa-un") se da cuando una clase usa métodos o instancias de otra sin que esta forme parte de su estructura, como cuando un método recibe un objeto como parámetro. La clave está en los incisos c y f, que se contradicen entre sí. El principio de diseño ampliamente aceptado es preferir la composición sobre la herencia (favor composition over inheritance), porque genera menos acoplamiento y nos permite cambiar el comportamiento con mayor flexibilidad. Por eso, el inciso f es correcto y el c es falso.

Pregunta número 20

What is the result?

```
abstract class Animal {
    void speak() {
        System.out.println("Animals make noise");
    }
    public static void main(String[] args) {
        Animal animal = new Dog();
        animal.speak();
    }
}
class Dog extends Animal {
    protected void speak() {
        System.out.println("The Dog barks");
    }
}
public class Test {
    public static void main(String... strings) {
        Animal.main(strings);
    }
}
```

Seleccione una:

- a. Animals make noise
- b. A runtime exception is thrown
- c. Compilation fails
- d. The Dog barks

Respuesta: Lo primero que voy a hacer es checar que el código compile. Animal es abstracta, pero eso no impide que tenga un main y métodos concretos; lo único que no podemos hacer es instanciar directamente un Animal, y aquí no lo hacemos, ya que creamos un Dog. En Animal, speak() tiene acceso por defecto, y Dog lo sobrescribe como protected, que es un acceso más amplio, así que la sobrescritura es válida. Desde el main de Test se llama a Animal.main(strings); esto también es válido, ya que main es un método static común y corriente, y un arreglo de String es compatible con un parámetro varargs String... y viceversa. Al ejecutarse, la

referencia animal apunta a un Dog, por lo que el polimorfismo hace que se ejecute la versión de Dog y se imprime "The Dog barks".

Pregunta número 21

What is the result?

```
abstract class Animal {
    public void speak() {
        System.out.println("Animals make noise");
    }
    public static void main(String[] args) {
        Animal animal = new Dog();
        animal.speak();
    }
}

class Dog extends Animal {
    protected void speak() {
        System.out.println("The Dog barks");
    }
}

public class Test {
    public static void main(String... strings) {
        Animal.main(strings);
    }
}
```

Seleccione una:

- (●) a. Compilation fails.
- ( ) b. The Dog barks
- ( ) c. A runtime exception is thrown.
- ( ) d. Animals make noise

Respuesta: A simple vista, este código es casi idéntico al de la pregunta anterior, pero hay un cambio de una sola palabra que lo cambia todo: ahora speak() en

Animal es public. En Dog seguimos teniendo protected void speak(), y protected es más restrictivo que public. Como la regla de la sobrescritura prohíbe reducir la visibilidad del método heredado, el compilador marca un error indicando que speak() en Dog no puede sobrescribir speak() en Animal porque intenta asignar privilegios de acceso más débiles. Por lo tanto, la compilación falla y no se imprime nada. Este tipo de preguntas me recuerda que debemos revisar con lupa los modificadores de acceso antes de calcular cualquier salida.

Pregunta número 22

Dado el siguiente código. ¿Cuál será el resultado?

```
import java.util.*;

class Alpha{
    public String mensaje (String msj) {
        return msj;
    }
}
class Beta extends Alpha {
    public String mensaje (String msj) {
        return msj.replace('a', 'e');
    }
}
class Gamma extends Beta{
    public String mensaje (String msj) {
        return msj.substring(2);
    }
}
public class MultiHerencia {
    public static void main(String[] args) {
        List<Alpha>list = new ArrayList<Alpha>();
        list.add(new Alpha());
        list.add(new Beta());
        list.add(new Gamma());
        for (Alpha a: list) {
            System.out.println(a.mensaje("Java"));
        }
    }
}
```

Seleccione una:

- a. Java Jeve ve
- b. Java Java Java
- c. Java Jeve va
- d. Compilation fails

Respuesta: Lo primero que noté en este ejercicio es que tenemos una jerarquía de tres niveles, y cada clase sobrescribe el método mensaje(). En el main creamos una lista de tipo Alpha, y como Beta y Gamma son subclases de Alpha, podemos agregar objetos de las tres clases. Al recorrer la lista, cada llamada a a.mensaje("Java") ejecuta la versión del objeto real gracias al polimorfismo. Con el objeto Alpha, el método regresa el texto tal cual, así que se imprime "Java". Con el objeto Beta, se reemplazan todas las 'a' por 'e', así que "Java" se convierte en "Jeve". Con el objeto Gamma, se ejecuta su propia versión, que hace substring(2) sobre el texto original, quedando "va". Aquí está la trampa: Gamma hereda de Beta, pero como sobrescribe el método y no llama a super.mensaje(), el reemplazo de Beta no se aplica. Si lo hiciera, el resultado sería "ve", que es lo que intenta sugerir el inciso a.

Pregunta número 23

Consider the following Java Code: What will be the output of the above code?

```
class Animal {
    public void makeSound() {
        System.out.println("Sound");
    }
    public void makeSound(String sound) {
        System.out.println(sound);
    }
}
class Dog extends Animal {
    @Override
    public void makeSound() {
        System.out.println("Bark");
    }
    public void makeSound(int times) {
        for (int i = 0; i < times; i++) {
            System.out.println("Bark");
        }
    }
}
public class Test {
    public static void main(String[] args) {
        Animal animal = new Animal();
        Animal dog = new Dog();
        animal.makeSound();
        animal.makeSound("Roar");
        dog.makeSound();
        dog.makeSound("Growl");
    }
}
```

Seleccione una:

- a. Sound Roar Bark Roard
- b. Sound Roar Bark Growl
- c. Sound Roar Bark Bark
- d. Sound Roar Sound Growl

Respuesta: En este ejercicio se combinan la sobrescritura y la sobrecarga. Animal tiene dos versiones de makeSound(): una sin parámetros y otra que recibe un String. Dog sobrescribe la versión sin parámetros y agrega una nueva sobrecarga que recibe un int. Hay que revisar las llamadas para entender lo que está sucediendo. animal.makeSound() es un objeto Animal, así que imprime "Sound", y animal.makeSound("Roar") imprime el texto recibido, "Roar". Luego, dog.makeSound() se ejecuta sobre un objeto Dog, y como Dog sobrescribe ese método, se imprime "Bark". Finalmente, dog.makeSound("Growl") busca un método que reciba un String; Dog no sobrescribió esa versión, así que se usa la heredada de Animal, que imprime "Growl". La salida completa es Sound, Roar, Bark, Growl. La sobrecarga con int en Dog nunca se llama y, de hecho, ni siquiera sería visible desde una referencia de tipo Animal.

Pregunta número 24

What is the result?

```
class Feline {
    public String type = "f";

    public Feline() {
        System.out.print("feline ");
    }
}

public class Cougar extends Feline {
    public Cougar() {
        System.out.print("cougar ");
    }

    void go() {
        type = "c";
        System.out.print(this.type + super.type);
    }

    public static void main(String[] args) {
        new Cougar().go();
    }
}
```

Seleccione una:

- a. Cougar c f
- b. Compilation fails
- c. Feline cougar c c
- d. Feline cougar c f

Respuesta: Lo primero que me gustaría mencionar de este ejercicio es que, al ejecutar new Cougar(), primero se ejecuta el constructor de Feline, que imprime "feline ", y después el de Cougar, que imprime "cougar ". Luego se llama a go(). Aquí está la clave: Cougar no declara su propio atributo type, simplemente hereda el de Feline. Por lo tanto, solo existe una variable type en el objeto, y tanto this.type como super.type hacen referencia exactamente a esa misma variable. Cuando go() ejecuta type = "c", cambia el valor de ese único atributo, así que this.type y super.type valen "c". La concatenación da "cc" y la salida completa es "feline cougar cc". El inciso d sería el resultado si Cougar declarara su propio String type, porque en ese caso tendríamos dos variables distintas, una ocultando a la otra (shadowing). El código compila sin problemas, porque type es public y es completamente accesible desde la subclase.

Pregunta número 25

¿Cuál de las siguientes anotaciones es verdadera acerca de la siguiente jerarquía de clases?

```
class Vehicle {
    String type = "Vehicle";
}
class Car extends Vehicle {
    String type = "Car";
}
class ElectricCar extends Car {
    String type = "ElectricCar";
}
```

Seleccione una:

- a. ElectricCar myCar = new ElectricCar(); System.out.println(myCar.type); // Salida: ElectricCar
- b. Car myCar = new ElectricCar(); System.out.println(myCar.type); // Salida: Vehicle
- c. ElectricCar myCar = new ElectricCar(); System.out.println(myCar.type); // Salida: Vehicle
- d. ElectricCar myCar = new ElectricCar(); System.out.println(myCar.type); // Salida: Car

Respuesta: En este ejercicio, cada clase declara su propio atributo type, por lo que la subclase oculta (shadowing) el atributo de la superclase en lugar de sobrescribirlo. A diferencia de los métodos, los atributos no son polimórficos: el valor que obtenemos depende del tipo de la referencia, no del tipo real del objeto. Con eso en mente, revisemos los incisos. En el inciso a, la referencia es de tipo ElectricCar, así que myCar.type accede al atributo de ElectricCar y se imprime "ElectricCar", lo cual coincide con el comentario. En el inciso b, la referencia es de tipo Car, así que se imprimiría "Car", no "Vehicle". Los incisos c y d usan una referencia ElectricCar, pero dicen que la salida sería "Vehicle" o "Car", lo cual es

incorrecto. La regla que hay que recordar aquí es que los métodos se resuelven por el objeto y los atributos por la referencia.

Pregunta número 26

What is the result?

```
11. class Person {
12. String name = "No name";
13. public Person (String nm) { name = nm; }
14. }
15.
16. class Employee extends Person {
17.  String empID = "0000";
18.  public Employee (String id) { empID = id; }
19. }
20.
21. public class EmployeeTest {
22.  public static void main(String[] args) {
23.   Employee e = new Employee("4321");
24.   System.out.println(e.empID);
25.  }
26. }
```

Seleccione una:

- a. 4321.
- b. 0000.
- c. An exception is thrown at runtime.
- d. Compilation fails because of an error in line 18.

Respuesta: El detalle clave de este ejercicio está en la clase Person. Al declarar el constructor Person(String nm), Java ya no genera el constructor por defecto sin argumentos, ya que este solo se crea cuando la clase no tiene ningún constructor declarado. Ahora pasemos a Employee. Su constructor Employee(String id) no llama explícitamente a super(...), así que el compilador inserta automáticamente un super() sin argumentos como primera línea. Ese super() intenta llamar a Person(), que no existe, por lo que la compilación falla precisamente en la línea 18, donde está el constructor de Employee. Para corregirlo, podríamos llamar a super("algún nombre") en la primera línea del constructor de Employee o agregar un constructor sin argumentos en Person. Como el código no compila, nunca llega a imprimirse "4321" ni "0000".

Pregunta número 27

What is the result?

```
class Atom {
    Atom() { System.out.print("atom "); }
}
class Rock extends Atom {
    Rock(String type) { System.out.print(type); }
}
public class Mountain extends Rock {
    Mountain() {
        super("granite ");
        new Rock("granite ");
    }
    public static void main(String[] a) { new Mountain(); }
}
```

Seleccione una:

- a. Compilation fails.
- b. Atom granite.
- c. Granite granite.
- d. Atom granite granite.
- e. An exception is thrown at runtime.
- f. Atom granite atom granite.

Respuesta: En este ejercicio hay que seguir la construcción paso a paso para entender qué está sucediendo. Al ejecutar new Mountain(), el constructor de Mountain llama a super("granite "), que ejecuta el constructor de Rock que recibe un String. Pero antes de que Rock ejecute su cuerpo, se llama implícitamente a super(), es decir, al constructor de Atom, que imprime "atom ". Después, Rock imprime el texto recibido, "granite ". Ahí termina el super(), y continúa el constructor de Mountain con la línea new Rock("granite "). Esta línea crea un objeto Rock completamente nuevo y, de nuevo, primero se ejecuta el constructor de Atom, que imprime "atom ", y luego Rock imprime "granite ". La salida completa es "atom granite atom granite". El inciso d es la trampa para quien olvide que el segundo objeto Rock también ejecuta el constructor de Atom, porque cada objeto nuevo recorre toda su cadena de constructores.

Pregunta número 28

What is the result?

```
class X {
    X() {
        System.out.print(1);
    }
    X(int x) {
        this();
        System.out.print(2);
    }
}

public class Y extends X {
    Y() {
        super(6);
        System.out.print(3);
    }
    Y(int y) {
        this();
        System.out.print(4);
    }

    public static void main(String[] a) {
        new Y(5);
    }
}
```

Seleccione una:

- a. 1234
- b. 2143
- c. 4321
- d. 134
- e. 13
- f. 2134

Respuesta: Esta pregunta es un buen ejercicio para seguir el encadenamiento de constructores con this() y super(). Empezamos con new Y(5), que llama al constructor Y(int y). Su primera línea es this(), que salta al constructor Y() sin argumentos. Ese constructor, a su vez, empieza con super(6), que llama a X(int x). Y este también empieza con this(), que llama a X(), el cual finalmente imprime 1. Ahora vamos de regreso: X(int x) continúa e imprime 2; después regresamos a Y(), que imprime 3; y finalmente regresamos a Y(int y), que imprime 4. La salida es

1234. Recordemos que this() y super() deben ser siempre la primera instrucción de un constructor, y por eso no se pueden usar los dos en el mismo constructor; aquí cada constructor usa solo uno, así que todo compila. La forma más fácil de resolver este tipo de ejercicios es seguir las llamadas hacia arriba y después imprimir de regreso.

Pregunta número 29

What should statement1, statement2, and statement3, be respectively, in order to produce the result?

Shape: constructor
Shape: foo
Square: foo

```
public class SuperTest {
    public static void main(String[] args) {
        //statement1
        //statement2
        //statement3
    }
}
class Shape {
    public Shape() {
        System.out.println("Shape: constructor");
    }
    public void foo() {
        System.out.println("Shape: foo");
    }
}
class Square extends Shape {
    public Square() {
        super();
    }
    public Square(String label) {
        System.out.println("Square: constructor");
    }
    public void foo() {
        super.foo();
    }
    public void foo(String label) {
        System.out.println("Square: foo");
    }
}
```

Seleccione una:

- a. Square square = new Square ("bar"); square.foo ("bar"); square.foo();
- b. Square square = new Square ("bar"); square.foo ("bar"); square.foo ("bar");
- c. Square square = new Square (); square.foo (); square.foo(bar);
- d. Square square = new Square (); square.foo (); square.foo("bar");
- e. Square square = new Square (); square.foo (); square.foo ();

Respuesta: Lo que debemos tener en cuenta en esta pregunta es que necesitamos producir tres líneas en orden: "Shape: constructor", "Shape: foo" y "Square: foo". Para la primera, debemos crear el objeto con new Square(), sin argumentos, porque ese constructor llama a super() y se imprime "Shape: constructor". Si usáramos new Square("bar"), también se ejecutaría el constructor de Shape por el super() implícito, pero además se imprimiría "Square: constructor", que no aparece en la salida esperada; esto descarta los incisos a y b. Para la segunda línea, llamamos a square.foo() sin argumentos, que ejecuta la versión de Square, y esta a su vez llama a super.foo(), imprimiendo "Shape: foo". Para la tercera, necesitamos la sobrecarga que recibe un String, es decir, square.foo("bar"), que imprime "Square: foo". Eso es justo lo que hace el inciso d. El inciso c no compila porque escribe foo(bar) sin comillas, y bar no es ninguna variable declarada. El inciso e llama dos veces a foo() sin argumentos, así que imprimiría "Shape: foo" dos veces.

Pregunta número 30

¿Cuál de los siguientes fragmentos de código generará una ClassCastException en tiempo de ejecución?

```
// Opción a
class A {}
class B extends A {}
public class Test {
    public static void main(String[] args) {
        A obj = new B();
        A a = (A) obj;
    }
}

// Opción b
class A {}
class B extends A {}
public class Test {
    public static void main(String[] args) {
        B obj = new B();
        A a = (A) obj;
    }
}

// Opción c
class A {}
class B extends A {}
public class Test {
    public static void main(String[] args) {
        A obj = new B();
        B b = (B) obj;
    }
}

// Opción d
class A {}
class B extends A {}
public class Test {
    public static void main(String[] args) {
        A ob = new A();
        B b = (B) ob;
    }
}
```

Seleccione una:

- a. a
- b. b
- c. c
- d. d

Respuesta: Lo primero que tenemos que recordar en este ejercicio es que una ClassCastException ocurre en tiempo de ejecución cuando intentamos convertir un

objeto a una clase de la que realmente no es instancia. El compilador permite el cast porque los tipos están relacionados por herencia, pero la JVM verifica el tipo real del objeto al ejecutar. Vayamos a revisar cada opción para ver qué está pasando. En la opción a, el objeto es un B y lo convertimos a A; como B es un A, no hay problema. En la opción b pasa lo mismo, ya que convertimos un B a A, lo cual es un upcasting siempre seguro. En la opción c, la referencia es de tipo A, pero el objeto real es un B, así que el cast a B funciona. La opción d es la que falla: creamos un objeto con new A(), y aunque el compilador acepta el cast a B, en tiempo de ejecución la JVM detecta que un A puro no es un B y lanza la ClassCastException. La regla es que un downcasting solo es seguro si el objeto realmente fue creado como la subclase.

Pregunta número 31

Which three are valid? (Choose three).

```
class ClassA{}
class ClassB extends ClassA{}
class ClassC extends ClassA{}
And:
ClassA p0 = new ClassA();
ClassB p1 = new ClassB();
ClassC p2 = new ClassC();
ClassA p3 = new ClassB();
ClassA p4 = new ClassC();
```

Seleccione una o más de una:

- [ ] a. p1 = p2;
- [x] b. p2 = (ClassC)p4;
- [ ] c. p2 = p4;
- [x] d. p1 = (ClassB)p3;
- [x] e. p0 = p1;
- [ ] f. p2 = (ClassC)p1;

Respuesta: La clave de este ejercicio es entender la jerarquía, la cual pienso yo que es relativamente sencilla de entender: ClassB y ClassC heredan de ClassA, pero son hermanas, es decir, no tienen relación entre sí. Revisemos cada opción para entender bien qué está sucediendo. El inciso b, p2 = (ClassC)p4, es válido: p4 es de tipo ClassA, así que el downcasting a ClassC compila, y como el objeto real es un ClassC, también funciona en ejecución. El inciso d, p1 = (ClassB)p3, sigue la misma lógica: p3 es un ClassA que realmente apunta a un ClassB, así que el cast compila y se ejecuta sin problemas. El inciso e, p0 = p1, es un upcasting de ClassB a ClassA, que siempre es válido sin necesidad de cast. Los demás fallan en

compilación: en el inciso a, p1 = p2 intenta asignar un ClassC a un ClassB, y como son clases hermanas, no son compatibles; el inciso c, p2 = p4, intenta asignar un ClassA a un ClassC sin cast; y el inciso f, (ClassC)p1, intenta convertir un ClassB a ClassC, y el compilador sabe que eso nunca podría funcionar, ya que no hay relación entre ellas.

Pregunta número 32

Which is true?

```
5.  class Building { }
6.  public class Barn extends Building {
7.      public static void main(String[] args) {
8.          Building build1 = new Building();
9.          Barn barn1 = new Barn();
10.         Barn barn2 = (Barn) build1;
11.         Object obj1 = (Object) build1;
12.         String str1 = (String) build1;
13.         Building build2 = (Building) barn1;
14.     }
15. }
```

Seleccione una:

- a. If line 10 is removed, the compilation succeeds.
- b. If line 11 is removed, the compilation succeeds.
- c. If line 12 is removed, the compilation succeeds.
- d. If line 13 is removed, the compilation succeeds.
- e. More than one line must be removed for compilation to succeed.

Respuesta: Para resolver esta pregunta tenemos que distinguir entre los errores de compilación y los de ejecución. La línea 10 hace un cast de un Building a Barn. Como Barn hereda de Building, el compilador acepta el cast; fallaría en ejecución con una ClassCastException, pero la pregunta solo habla de compilar. La línea 11 convierte a Object, lo cual siempre es válido. La línea 13 hace un upcasting de Barn a Building, que también es válido. El verdadero problema está en la línea 12: (String) build1. String y Building no tienen ninguna relación de herencia, y además String es una clase final, así que el compilador sabe que es imposible que un Building sea un String y marca un error de tipos inconvertibles. Por lo tanto, si eliminamos la línea 12, la compilación tendrá éxito. Los otros incisos proponen eliminar líneas que no causan ningún error de compilación.

Pregunta número 33

Dado el siguiente código: ¿Cuál es el resultado de la ejecución del programa?

```
interface Omega {}

class Gamma {}

class Epsilon extends Gamma implements Omega {}

public class Test {
    public static void main(String[] args) {
        Gamma g = new Epsilon();
        Omega o = (Omega) g;  // Corrected line
        Epsilon e = (Epsilon) g; // Corrected line
        Omega m= e;
    }
}
```

Seleccione una:

- a. An exception is thrown at runtime.
- b. No option is correct.
- c. Compilation fails.
- d. The code runs with no output.

Respuesta: Primero hay que revisar la estructura. Epsilon hereda de Gamma e implementa la interfaz Omega. En el main, creamos un Epsilon y lo guardamos en una referencia de tipo Gamma. Luego hacemos el cast (Omega) g. Gamma no implementa Omega, pero como Gamma no es una clase final, el compilador acepta el cast, ya que alguna subclase de Gamma podría implementar Omega. En tiempo de ejecución, el objeto real es un Epsilon, que sí implementa Omega, así que el cast funciona. El siguiente cast (Epsilon) g es un downcasting que también es correcto, ya que el objeto es realmente un Epsilon. Finalmente, Omega m = e es un upcasting implícito, porque Epsilon implementa Omega. Todo compila y se ejecuta sin errores, pero como no hay ninguna instrucción de impresión, el programa simplemente termina sin mostrar nada.

Pregunta número 34

Cuál es el resultado?

```
import java.util.ArrayList;
public class Test {
    public static void main(String[] args) {
        String v = new String();
        StringBuffer sb = new StringBuffer();
        class MyList extends ArrayList {
        }
        System.out.println((v instanceof String) &&
            (sb instanceof Object) && (new MyList() instanceof ArrayList));
    }
}
```

Seleccione una:

- a. false
- b. true
- c. The code runs with no output.
- d. Compilation fails.
- e. An exception is thrown at runtime.
- f. null

Respuesta: Aquí tenemos tres evaluaciones de instanceof unidas por el operador &&, y además una clase local declarada dentro del main. Revisemos primero que el código compile. Declarar una clase local (MyList) dentro de un método es completamente válido, y como extiende de ArrayList sin genéricos, el compilador solo lanza advertencias. Ahora evaluemos las condiciones. v es un String, así que v instanceof String es true. sb es un StringBuffer, y todas las clases heredan de Object, así que sb instanceof Object también es true. Finalmente, new MyList() crea un objeto de la clase local, que hereda de ArrayList, así que el instanceof también regresa true. Como las tres condiciones son verdaderas, el resultado del && es true, y eso es lo que se imprime. El inciso f es incorrecto porque instanceof siempre regresa un boolean, nunca null.

Pregunta número 35

What is the result?

```
class Foo{
    public void addFive(){ a += 5; System.out.println("f");}
}
class Bar extends Foo{
    public int a = 8;
    public void addFive(){ this.a += 5; System.out.println("b");}
}
Invoked with:
    Foo f = new Bar();
    f.addFive();
    System.out.println(f.a);
```

Seleccione una:

- a. b 8
- b. b 13
- c. f 13
- d. An exception is thrown at runtime.
- e. b 3
- f. f 8
- g. Compilation fails
- h. f 3

Respuesta: Lo primero que se me ocurrió en este ejercicio es aventurarme a intentar calcular el valor de a, pensando que podría llegar a "b 13", pero más temprano que tarde caí en cuenta de que el código ni siquiera compila. El problema está en la clase Foo: su método addFive() intenta ejecutar a += 5, pero Foo no tiene ningún atributo llamado a. El atributo a se declara hasta la clase Bar, y una superclase no puede ver los atributos que se definen en sus subclases. Por eso, el compilador marca un error de símbolo no encontrado. Además, hay un segundo problema en System.out.println(f.a): la referencia f es de tipo Foo, y el compilador busca el atributo en el tipo de la referencia, no en el objeto, así que tampoco lo encuentra. Si Foo tuviera su propio atributo a, entonces sí podríamos analizar la salida, y tendríamos que recordar que los atributos no son polimórficos.

Pregunta número 36

Which two, independently, will allow Sub to compile? (Choose two)

```
1.  class Super {
2.      private int a;
3.      protected Super(int a) { this.a = a; }
4.  }
...
11. class Sub extends Super {
12.     public Sub(int a) { super(a); }
13.     public Sub() { this.a = 5; }
14. }
```

Seleccione una o más de una:

- [ ] a. Change line 2 to: public int a;
- [x] b. Change line 13 to: public Sub() { super(5); }
- [ ] c. Change line 2 to: protected int a;
- [x] d. Change line 13 to: public Sub() { this(5); }
- [ ] e. Change line 13 to: public Sub() { super(a); }

Respuesta: En este ejercicio encontramos que el problema original está en la línea 13: el constructor Sub() no llama a ningún super(...), así que el compilador inserta un super() sin argumentos, y Super no tiene un constructor sin argumentos; solo tiene Super(int a). Además, en esa misma línea se hace this.a = 5, pero a es private en Super, así que tampoco es accesible desde Sub. Ahora toca revisar qué opciones arreglan el código. El inciso b cambia la línea 13 a super(5), con lo que llamamos al constructor correcto y eliminamos el acceso directo a `a`; esto compila. El inciso d cambia la línea 13 a this(5), que llama a Sub(int a), y este sí llama a super(a), así que también compila. Los incisos a y c no funcionan, porque aunque hacen que el atributo sea accesible, el problema del super() implícito sigue existiendo. El inciso e falla porque a es private en Super y, además, no podemos usar un atributo de instancia como argumento de super() antes de que el objeto termine de construirse.

Pregunta número 37

Which five methods, inserted independently at line 5, will compile? (Choose five).

```
1 public class Blip{
2       protected int blipvert(int x){ return 0 ;}
3 }
4 class Vert extends Blip{
5       //insert code here
6 }
```

Seleccione una o más de una:

- [x] a. protected long blipvert(long x) { return 0; }
- [x] b. public int blipvert(int x) { return 0; }
- [x] c. private int blipvert(long x) { return 0; }
- [x] d. protected long blipvert(int x, int y) { return 0; }
- [ ] e. protected long blipvert(int x) { return 0; }
- [x] f. protected int blipvert(long x) { return 0; }
- [ ] g. private int blipvert(int x) { return 0; }

Respuesta: En este ejercicio para encontrar los cinco métodos que compilan, tenemos que distinguir entre sobrecarga y sobrescritura. Si la lista de parámetros es distinta de (int x), es una sobrecarga, y en ese caso no importan ni el tipo de retorno ni el modificador de acceso. Por eso, los incisos a (long x), c (long x), d (int x, int y) y f (long x) compilan sin problema, aunque tengan acceso private o retornen long, ya que todos son métodos nuevos. Si la lista de parámetros es exactamente (int x), entonces es una sobrescritura y debemos respetar las reglas: mismo tipo de retorno (o un subtipo) y el mismo acceso o uno más amplio. El inciso b cumple con esto, porque mantiene el retorno int y amplía el acceso de protected a public. El inciso e falla porque cambia el tipo de retorno de int a long, y no es un retorno compatible. El inciso g falla porque private reduce la visibilidad del método heredado.

Pregunta número 38

Which three lines will compile and output "Right on!"?

```
13.  public class Speak {
14.      public static void main(String[] args) {
15.          Speak speakIT = new Tell();
16.          Tell tellIt = new Tell();
17.          speakIT.tellItLikeItIs();
18.          (Truth) speakIT.tellItLikeItIs();
19.          ((Truth) speakIT).tellItLikeItIs();
20.          tellIt.tellItLikeItIs();
21.          (Truth) tellIt.tellItLikeItIs();
22.          ((Truth) tellIt).tellItLikeItIs();
23.      }
24.  }

class Tell extends Speak implements Truth {
    @Override
    public void tellItLikeItIs() {
        System.out.println("Right on!");
    }
}

interface Truth {
    public void tellItLikeItIs();
}
```

Seleccione una o más de una:

- [ ] a. Line 17
- [ ] b. Line 18
- [x] c. Line 19
- [x] d. Line 20
- [ ] e. Line 21
- [x] f. Line 22

Respuesta: En este ejercicio tenemos que fijarnos en dos cosas: el tipo de la referencia y la precedencia del cast. speakIT es una referencia de tipo Speak, aunque apunta a un objeto Tell, mientras que tellIt es una referencia de tipo Tell. La línea 17 no compila, porque el compilador busca el método tellItLikeItIs() en Speak y no lo encuentra. La línea 18 tampoco compila por un tema de precedencia: el punto (.) tiene mayor precedencia que el cast, así que primero se intenta llamar al método sobre speakIT, y no existe. Además, aunque existiera, estaríamos haciendo un cast de un resultado void, lo cual no es una instrucción válida. La línea 19 sí compila,

porque con los paréntesis adicionales primero convertimos speakIT a Truth, y luego llamamos al método. La línea 20 compila porque Tell tiene el método. La línea 21 falla por el mismo problema de precedencia que la 18, ya que intenta hacer un cast al resultado void del método. Finalmente, la línea 22 compila e imprime "Right on!".

Pregunta número 39

Dado el siguiente código. ¿Cuál es el resultado?

```
class A{
    int x = 8;
    public String ejemplo() {
        x+=2;
        return "Valor de la clase A";
    }
}
class B extends A{
    int x = 12;
    public String ejemplo() {
        x+=4;
        return "Valor de la clase B";
    }
}
class C extends B{
    int x = 18;
    public String ejemplo() {
        x+=6;
        return "Valor de la clase C";
    }
}
public class HerenciaVar extends C{
    public static void main(String[] args) {
        A ac = new C();
        System.out.println(ac.ejemplo());
        System.out.println(ac.x + 20);
    }
}
```

Seleccione una:

- a. Valor de la clase A 30
- b. Valor de la clase B 36
- c. Valor de la clase C 28
- d. Valor de la clase C 44

Respuesta: Esta pregunta pone a prueba una de las reglas más importantes del polimorfismo: los métodos son polimórficos, pero los atributos no. La referencia ac es de tipo A, pero el objeto real es un C. Cuando llamamos a ac.ejemplo(), se ejecuta la versión de C por el polimorfismo; ese método incrementa el atributo x de C (de 18 a 24) y regresa "Valor de la clase C". En la siguiente línea, ac.x accede al atributo, y como los atributos se resuelven según el tipo de la referencia, obtenemos el x de la clase A, que sigue valiendo 8, ya que el método que se ejecutó modificó el x de C, no el de A. Entonces, ac.x + 20 da 28. El inciso d sería el resultado si pensáramos que ac.x se refiere al x de C (24 + 20 = 44), y los incisos a y b son para quien piense que se ejecuta el método de A o de B.

Pregunta número 40

What is the result?

```
abstract class Animal {
    private void speak() {
        System.out.println("Animals make noise");
    }
    public static void main(String[] args) {
        Animal animal = new Dog();
        animal.speak();
    }
}
class Dog extends Animal {
    protected void speak() {
        System.out.println("The Dog barks");
    }
}
public class Test {
    public static void main(String... strings) {
        Animal.main(strings);
    }
}
```

Seleccione una:

- a. Animals make noise
- b. Compilation fails.
- c. The Dog barks
- d. A runtime exception is thrown.

Respuesta: Este es otro ejercicio parecido a los que vimos en las preguntas 20 y 21, pero con un cambio en el modificador de acceso: ahora speak() es private en Animal. Los métodos private no se heredan, así que el speak() de Dog no es una

sobrescritura, sino un método completamente nuevo e independiente. Por esa misma razón, no aplica la regla de no reducir la visibilidad, y el código compila sin problemas. En el main de Animal, la llamada animal.speak() se resuelve en tiempo de compilación a partir del tipo de la referencia, que es Animal. Como estamos dentro de la propia clase Animal, su método private es accesible, y como no hay sobrescritura, tampoco hay polimorfismo. Se ejecuta el método de Animal y se imprime "Animals make noise", aunque el objeto real sea un Dog.